General Generative AI

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

KAUST Academy
King Abdullah University of Science and Technology

April 27, 2026

Deep Unsupervised Learning (DUL): **Motivation**

Motivation — Why Go Beyond Supervised Learning?

Supervised learning is effective, but it has some limitations:

- Needs a lot of labeled data to work well.
- Getting labels can be difficult or costly.
- Depends on experts to label the data.
- Not practical for many real-world situations where labeling is hard or impossible.

The Bottleneck

- Humans are good at:
  - Learning patterns from experience
  - Discovering hidden structures
- But:
  - They are slow and limited
  - Not always available
  - Can't always explain what they observe

A New Possibility

- What if machines could learn to:
  - Find hidden patterns
  - Understand structures in data
  - Group and compare similar things

Without any human-provided labels?

Unsupervised Learning to the Rescue

- Learns **without labels**
- Discovers **structure within the data**
- Mimics how humans learn from **observation**
- Useful for:
  - Clustering
  - Dimensionality reduction
  - Representation learning

Why Deep?

- Simple models can't handle:
  - High variability in real-world data
  - Complex abstract features
- We need:
  - Neural networks
  - With many layers $\rightarrow$ **"Deep"**
  - To learn hierarchical patterns

Deep Unsupervised Learning — The Goal

- Discover hidden patterns
- Learn rich representations
- Group and distinguish data points
- Match similar structures

Deep Unsupervised Learning (DUL): **Introduction**

What is Deep Unsupervised Learning?

Deep Unsupervised Learning aims to discover **hidden patterns** and **structures** in raw data using deep neural networks, **without any human-provided labels**.

It focuses on **capturing rich patterns in raw data with deep networks in a label-free way**.

- **Learns from observation** — mimics how humans learn from experience
- **Discovers structure** — finds groups, similarities, and representations
- **Scales to complex data** — uses deep models to capture abstract features
- **Generative Models** — recreate raw data distribution
- **Self-supervised Learning** — "puzzle" tasks that require semantic understanding

Why do we care?

Geoffrey Hinton
(in his 2014 AMA on Reddit)

"The brain has about 10^{14} *synapses and we only live for about* 10^{9} *seconds. So we have a lot more parameters than data. This motivates the idea that we must do a lot of unsupervised learning since the perceptual input (including proprioception) is the only place we can get* 10^{5} *dimensions of constraint per second."*

Yann LeCun

Need tremendous amount of information to build machines that have common sense and generalize well.

[LeCun-20161205-NeurIPS-keynote]

- **"Pure" Reinforcement Learning (cherry)**
  - The machine predicts a scalar reward given once in a while.
  - **A few bits for some samples**
- **Supervised Learning (icing)**
  - The machine predicts a category or a few numbers for each input
  - Predicting human-supplied data
  - **10→10,000 bits per sample**
- **Unsupervised/Predictive Learning (cake)**
  - The machine predicts any part of its input for any observed part.
  - Predicts future frames in videos
  - **Millions of bits per sample**

- **(Yes, I know, this picture is slightly offensive to RL folks. But I'll make it up)**

"Ideal Intelligence"

- **"Ideal Intelligence" is all about compression** (finding all patterns)
- Finding all patterns = short description of raw data (*low Kolmogorov Complexity*)
- Shortest code-length = optimal inference (*Solomonoff Induction*)
- Extensible to optimal action-making agents (*AIXI*)

Transfer Learning via Compression:

- Assume we pretrain unsupervised on Data Distribution D_1 and then finetune on Data Distribution D_2
- If D_1 and D_2 are related, compressing D_2 conditioned on D_1 should be more efficient than compressing D_2 outright
- **Hence:** pretraining on D_1 should aid faster learning of D_2

Autoencoders

- Family of neural networks for which the input is the same as the output. They work by compressing the input into a latent-space representation, and then reconstructing the output from this representation.
- The idea is to project the input into a latent space and then reconstruct the input from that latent space representation
- Consist of two parts: Encoder and decode.
  - **Encoder** projects the input to a latent space Z (compresses the input into a lower-dimensional representation).
  - **Decoder** takes the encoded embedding vector and reconstructs the input from it.
  - We also use altered versions of input as output which can be even more interesting.

Autoencoders (cont.)

- **Bottleneck**
  - The central, compressed layer that forces the network to learn the most important features.
  - Introduces a constraint to avoid learning a trivial identity function.
- **Loss Function**
  - Usually **Mean Squared Error (MSE)** between the input and the reconstructed output.
  - Other losses (e.g., binary cross-entropy, KL divergence) can be used depending on the application.

Autoencoders: Architecture

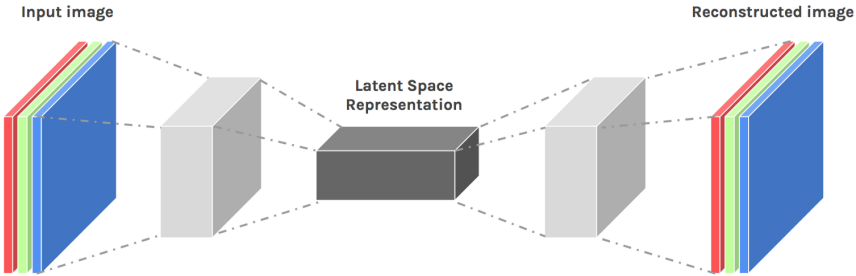


Figure 2: Autoencoder architecture

Autoencoders: Architecture (cont.)

Design Considerations:

- **Depth**: Deeper architectures can capture more abstract representations but may increase the risk of overfitting and require more training data.
- **Width**: Wider layers (more neurons) increase model capacity but can reduce compression effectiveness.
- **Symmetry**: Encoders and decoders are often symmetrical but do not have to be. Asymmetrical designs may be better suited for specific tasks.
- **Regularization**: Techniques such as dropout, weight decay (L1/L2), and sparsity constraints help reduce overfitting and improve generalization.
- **Latent Dimension Size**: A crucial parameter that determines how compressed the representation is. Too small may lose information; too large may not compress enough.
- **Activation Functions**: Nonlinearities like ReLU, sigmoid, or tanh allow the model to learn complex mappings. The final layer typically uses a sigmoid or linear activation depending on the output format.

Autoencoders: Architecture (cont.)

Component	Description
Encoder	Transforms the input data into a compressed latent representation using layers such as Dense, Convolutional, or Recurrent layers.
Latent Space (Bottleneck)	The central compact representation of the input. Forces the model to learn the most essential features.
Decoder	Reconstructs the input from the latent space. Typically mirrors the encoder's structure in reverse.
Loss Function	Measures the difference between input and output (e.g., Mean Squared Error, Binary Cross-Entropy). Drives learning during training.
Training	Involves minimizing reconstruction error using gradient-based optimization methods such as SGD or Adam.

Table 1: Architecture Summary Table

Autoencoders: Architecture (cont.)

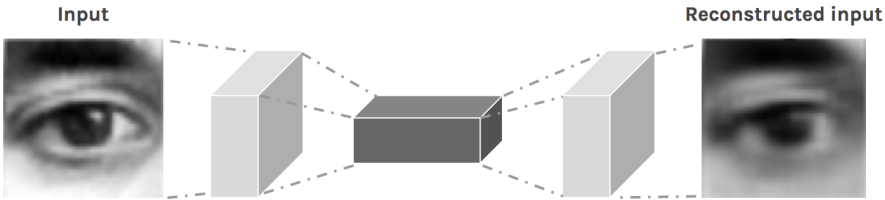


Figure 3: Sample Autoencoder

Autoencoders: Interactive Demo

https://douglasduhaime.com/posts/visualizing-latent-spaces.html

Autoencoders as Generative Models

- Autoencoders project data into a latent space Z.
- The latent space is not necessarily continuous or structured.
- *What if we sample a new embedding vector from Z and then have the decoder reconstruct the image from it?*
- **Does not work**. Autoencoders just learn a function that maps input to output. The learned latent space is too discontinous to work as a generative model.
- Sampling randomly from the latent space may result in invalid outputs.
- They do not maximize the likelihood of the data.

Autoencoders as Generative Models (cont.)

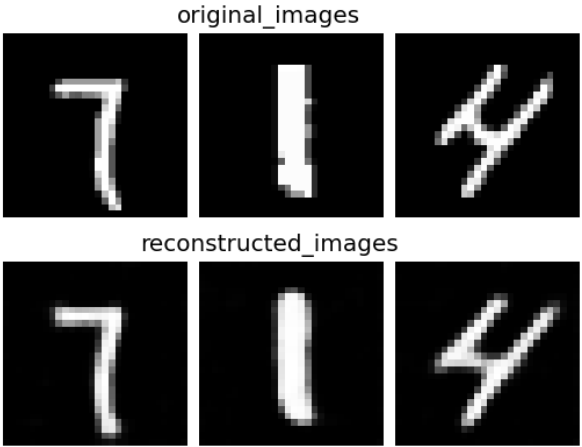


Figure 4: Image reconstruction with autoencoder trained on MNIST digits

Autoencoders as Generative Models (cont.)

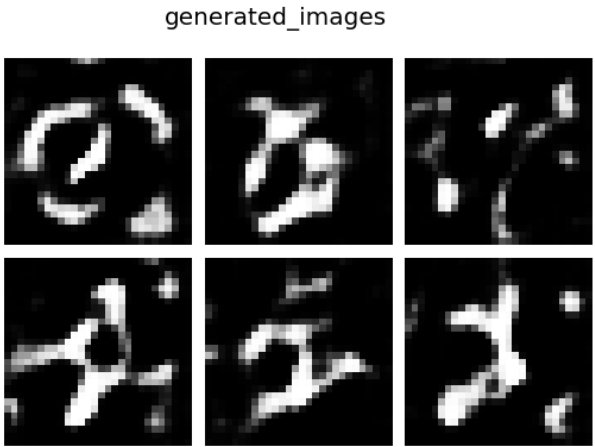
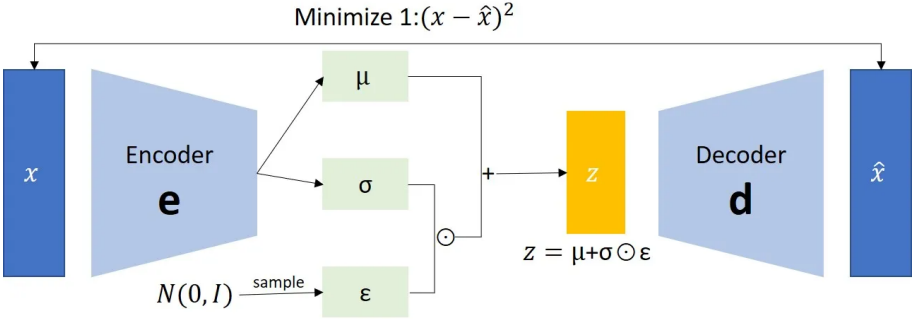


Figure 5: Image generation with autoencoder trained on MNIST digits. Encoding vector sampled from latent space Z and the passed to decoder.

Autoencoders as Generative Models (cont.)

Introduction: Variational Autoencoders (VAE)

- While autoencoders themselves have very low generative power, we will soon talk about a type of autoencoders called **Variational Autoencoders** which are specifically designed for generative modeling.
- **Variational Autoencoders (VAEs)**: Introduces a probabilistic framework where the encoder outputs parameters of a distribution (typically Gaussian). Sampling from this distribution and decoding allows generation of new data.
- Other use cases of Autoencoders include:
  - Data encoding and dimensionality reduction
  - Image denoising and super-resolution
  - Image completion
  - Image colorization



Minimize 2: $\frac{1}{2}\sum_{i=1}^{N}(\exp(\sigma_i) - (1+\sigma_i) + {\mu_i}^2)$

Why do we need VAEs?

- Traditional autoencoders are:
  - deterministic
  - lack generative capabilities
  - do not model uncertainty in latent space
- Need a compact, meaningful representation (latent space)
- We want to learn a probabilistic model of the data
- We want to generate new data samples similar to training data

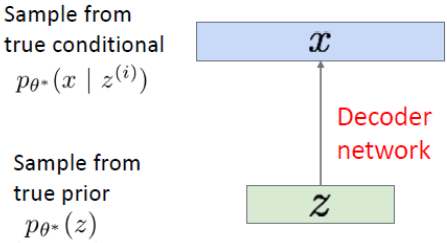
Motivation (cont.)

Applications

- Image generation (e.g., generating new faces or handwritten digits).
- Data compression and denoising.
- Anomaly detection in various domains.

VAE: **Introduction**

- We want a **generative model**, which given a prior **z** outputs a new sample from data.
- To make the latent space Z continuous, let's choose it to be Gaussian
- So now, we want to estimate the true parameters θ^* of this generative model.



- **Probabilistic Encoder:** Instead of encoding an input x to a fixed point z, the encoder learns a distribution over the latent variable z conditioned on the input:

$$q(z \mid x)$$

Typically, this is a multivariate Gaussian with parameters $\mu(x)$ and $\sigma(x)$ learned by a neural network.

- **Probabilistic Decoder:** Given a sample z from the latent distribution, the decoder reconstructs the input by modeling:

$$p(x \mid z)$$

This allows for generating new data by sampling from the latent space.

Introduction (cont.)

- **Latent Space:**
  - The model learns a smooth, continuous space for z.
  - Similar data points end up close together in this space.
  - This helps the model understand the main patterns in the data.

Objectives and Training: The training objective of a VAE is to maximize the **Evidence Lower Bound (ELBO)**:

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\mathrm{KL}}(q(z|x)\|p(z))$$

- The first term, $\mathbb{E}_{q(z|x)}[\log p(x|z)]$, encourages accurate reconstruction.
- The second term, $D_{\mathrm{KL}}(q(z|x)\|p(z))$, regularizes the latent space by making the approximate posterior $q(z|x)$ close to the prior $p(z)$, usually $\mathcal{N}(0, I)$.

Introduction (cont.)

Why Use VAEs?:

- Enable generative modeling: generate new samples by sampling from the latent distribution.
- Learn smooth, structured, and interpretable latent representations.
- Provide a principled probabilistic framework for inference and generation.

VAE: **Loss Function**

Loss Function Breakdown

Total Loss:

$$L = \text{Reconstruction Loss} + \text{KL Divergence}$$

Reconstruction Loss:

- Measures how well $\hat{x}$ matches x.
- Common choices: Mean Squared Error (MSE) or Binary Cross-Entropy.

KL Divergence:

- Measures how much $q(z \mid x)$ diverges from the prior $p(z)$.
- Encourages the latent space to follow a standard normal distribution.

VAE: **Results**

Results

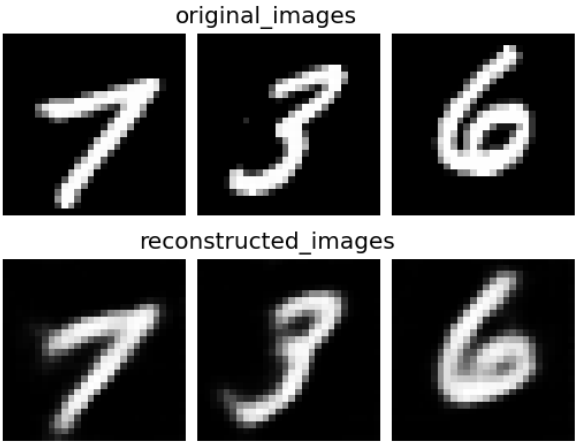


Image reconstruction with variational autoencoders on MNIST digits dataset

Results (cont.)

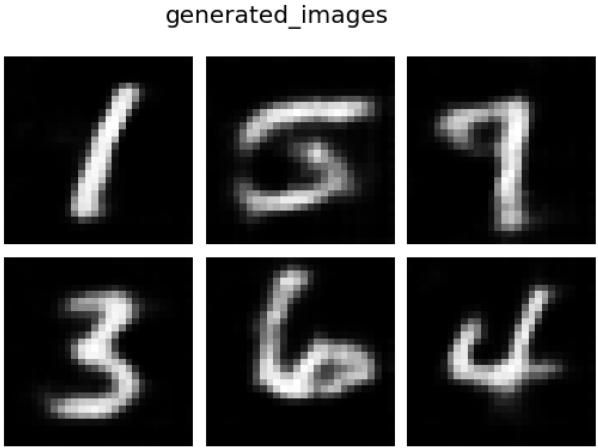
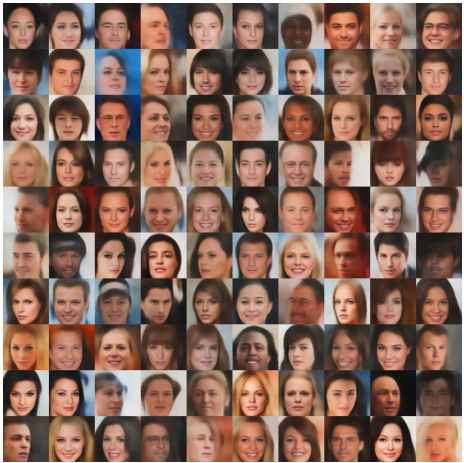


Image generation with variational autoencoders on MNIST digits dataset. Sample an encoding vector from $\mathcal{N}(0, 1)$ and passed it through decoder

Results (cont.)

Image generation with variational autoencoders on CIFAR-10 32x32 dataset

Results (cont.)



https://github.com/houxianxu/DFC-VAE

VAE: **Limitations and Challenges**

Limitations and Challenges

Limitations:

- **Gaussian Assumption:** The assumption that the latent variables follow a Gaussian distribution may not hold for all datasets.
- **Over-smoothing:** The model may produce overly smooth reconstructions, losing fine details in the data.
- **Sensitivity to Hyperparameters:** The performance of VAEs can be sensitive to the choice of hyperparameters, such as the weight of the KL divergence term.
- **Computational Complexity:** Training VAEs can be computationally expensive, especially for large datasets or complex models.
- **Evaluation Metrics:** Evaluating the quality of generated samples can be subjective and challenging, as traditional metrics may not capture the nuances of the data.

Limitations and Challenges (cont.)

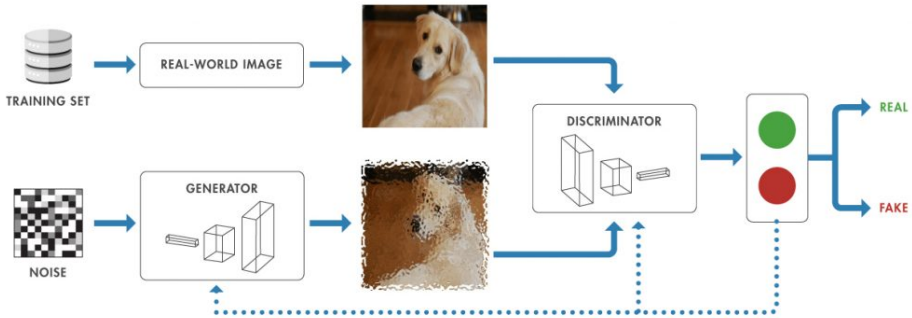
Challenges:

- **Training Instability:** VAEs can be difficult to train, especially with complex datasets.
- **Mode Collapse:** The model may generate samples from only a subset of the latent space.
- **Balancing Reconstruction and Regularization:** Finding the right balance between reconstruction loss and KL divergence can be tricky.
- **Posterior Collapse:** In some cases, the model may ignore the latent variables, leading to poor representations.
- **Limited Expressiveness:** The Gaussian assumption for the latent space may not capture complex data distributions.
- **Disentanglement:** Achieving disentangled representations can be challenging, especially in high-dimensional spaces.

Limitations and Challenges (cont.)

Future Directions:

- **Improved Training Techniques:** Developing better optimization methods to stabilize training.
- **Advanced Architectures:** Exploring more complex latent variable models, such as Normalizing Flows or Hierarchical VAEs.
- **Better Regularization Techniques:** Investigating alternative regularization methods to improve disentanglement and representation quality.
- **Hybrid Models:** Combining VAEs with other generative models (e.g., GANs) to leverage their strengths.
- **Application-Specific Variants:** Tailoring VAEs for specific applications, such as text or video generation.



Architecture of a Generative Adversarial Network (GAN). The generator creates fake data, while the discriminator distinguishes between real and fake data. The two networks are trained in opposition to each other, leading to improved performance over time.

GANs: Motivation

So far, we have studied generative models based on maximizing likelihood or its approximations:

1. **Autoregressive Models**: $p_\theta(x) = \prod_{i=1}^{N} p_\theta(x_i|x_{<i})$
2. **Variational Autoencoders (VAEs)**: $p_\theta(x) = \int_z p_\theta(x|z)p_\theta(z)$
3. **Normalizing Flow Models**: $p_X(x;\theta) = p_Z(f_\theta^{-1}(x)) \left|\det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|$

Problem with Traditional Generative Models:

- They often require complex approximations or sampling methods.
- They can produce blurry or unrealistic samples.
- They depend on explicit density estimation, which can be computationally expensive.

What if we give up on explicitly modeling density, and just want ability to sample?

GANs: Motivation (cont.)

Generative Adversarial Networks (GANs) provide a solution to this problem by introducing a novel approach to generative modeling:

- (introduced by Ian Goodfellow et al., 2014) do not require explicit density estimation or Markov chains.
- consist of two neural networks—a **generator** and a **discriminator**—in a 2-player (zero-sum) game.
- The adversarial setup enables the generator to learn complex data distributions and produce high-fidelity, realistic samples.

GAN Progress

- GANs are the most prominent example of Implicit Models.

GAN Progress (cont.)

https://www.youtube.com/watch?v=sW6D34mckkk

[BigGAN, Brock, Donahue, Simonyan, 2018]

GAN Progress (cont.)

$$\min_G \max_D \mathbb{E}_x[\log(D(x))] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

10/25/2018

Sold at Christies for $432,500

GANs: Objectives

1. **Understand the fundamental concepts and motivations behind GANs.**
   - 1.1 Learn why GANs were introduced and what problems they aim to solve in generative modeling.
2. **Explore the mathematical formulation and training process of GANs.**
   - 2.1 Study the minimax objective and the roles of the generator and discriminator.
   - 2.2 Analyze how GANs compare distributions via samples.
3. **Examine objective functions and training challenges.**
   - 3.1 Review standard and alternative objective functions.
   - 3.2 Identify common problems in GAN training, such as mode collapse and instability.
4. **Survey popular GAN variants and latent space concepts.**
   - 4.1 Wasserstein GANs, Conditional GANs, CycleGANs, StyleGANs.
   - 4.2 Understand the role and selection of latent spaces in GANs.

GANs: **Definition and Core Components**

Definition and Core Components

- Let $S_1 = D = \{x \sim p_{\text{data}}\}$ and $S_2 = \{x \sim p_\theta\}$.
- **Idea**: Train the generative model to minimize a two-sample test objective between S_1 and S_2, i.e., the generator.

Definition and Core Components

- Let $S_1 = D = \{x \sim p_{\text{data}}\}$ and $S_2 = \{x \sim p_\theta\}$.
- **Idea**: Train the generative model to minimize a two-sample test objective between S_1 and S_2, i.e., the generator.
- **Question**: How do we obtain a two-sample test objective?

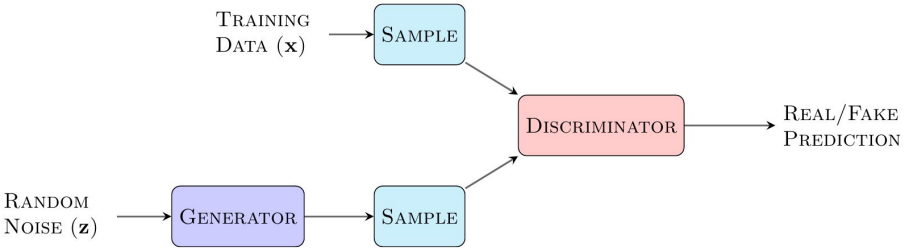
Definition and Core Components

- Let $S_1 = D = \{x \sim p_{\text{data}}\}$ and $S_2 = \{x \sim p_\theta\}$.
- **Idea**: Train the generative model to minimize a two-sample test objective between S_1 and S_2, i.e., the generator.
- **Question**: How do we obtain a two-sample test objective?
- **Another Idea**: Train another neural network to discriminate between the two samples, i.e., the discriminator.

Definition and Core Components

- Let $S_1 = D = \{x \sim p_{\text{data}}\}$ and $S_2 = \{x \sim p_\theta\}$.
- **Idea**: Train the generative model to minimize a two-sample test objective between S_1 and S_2, i.e., the generator.
- **Question**: How do we obtain a two-sample test objective?
- **Another Idea**: Train another neural network to discriminate between the two samples, i.e., the discriminator.
- And Voila! That's how we arrive at a GAN. The remaining step is to define the training method.

GANs: Definition and Core Components

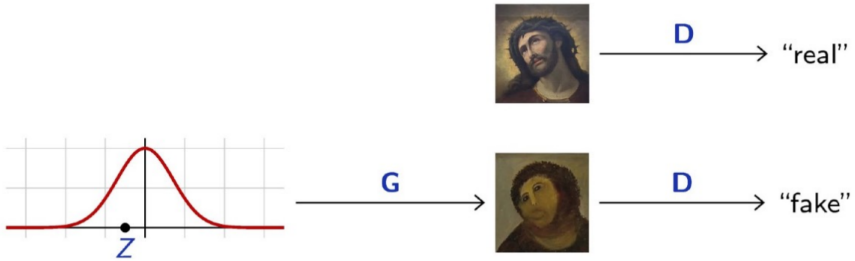


A GAN model diagram[1]

GANs: Definition and Core Components (cont.)

- Generative Adversarial Networks were introduced by Ian Goodfellow et al. (2014).
- The idea behind GANs is to train two networks jointly:
  - A **discriminator (D)** to classify samples as "real" or "fake".
  - A **generator (G)** to map a simple fixed distribution to samples that fool **D**.
- The approach is **adversarial** since the two networks have opposing objectives.

GANs: Definition and Core Components (cont.)



The generator transforms a simple distribution into samples resembling real data. The discriminator distinguishes between real and fake samples.

[1] https://github.com/JamesAllingham/LaTeX-TikZ-Diagrams

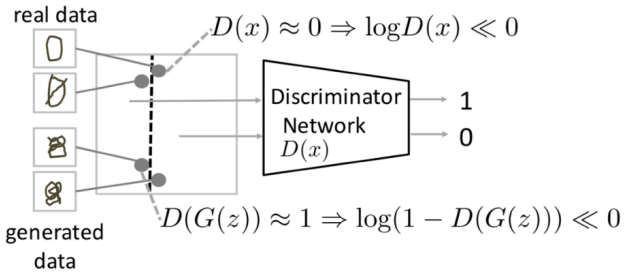
GANs: **Objective function**

Understanding the Objective function

The goal of GANs is to find a **Nash equilibrium** between the generator and discriminator. The generator tries to minimize the probability of the discriminator correctly classifying fake data.

$$\max_D(\mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z))])$$

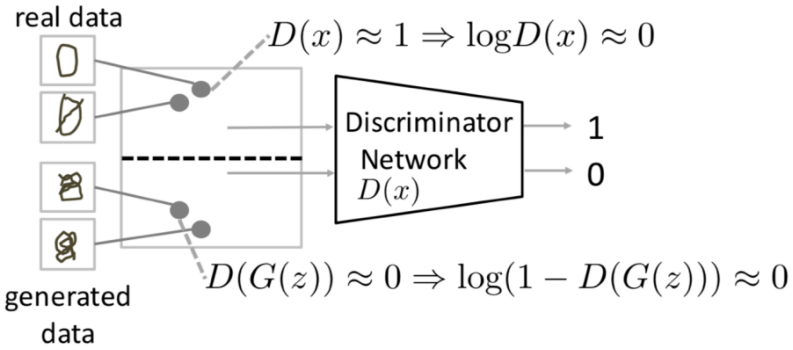
$D(x)$ should be 1 $D(G(z))$should be 0



Understanding the Objective function (cont.)

$$\max_D(\mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z))])$$

$D(x)$ should be 1 $D(G(z))$should be 0

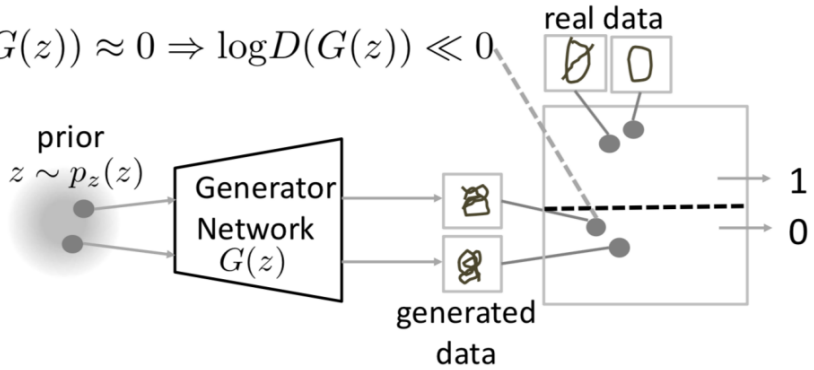


Understanding the Objective function (cont.)

$$\max_G(\mathbb{E}_{z\sim p_z(z)}[\log(D(G(z))])$$

$D(G(z))$ should be 1

$D(G(z)) \approx 0 \Rightarrow \log D(G(z)) \ll 0$

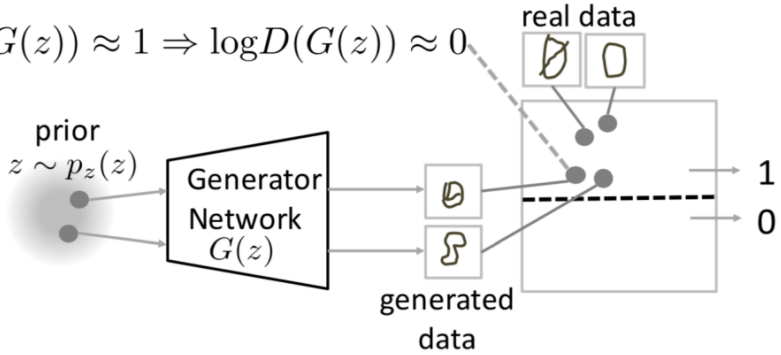


Understanding the Objective function (cont.)

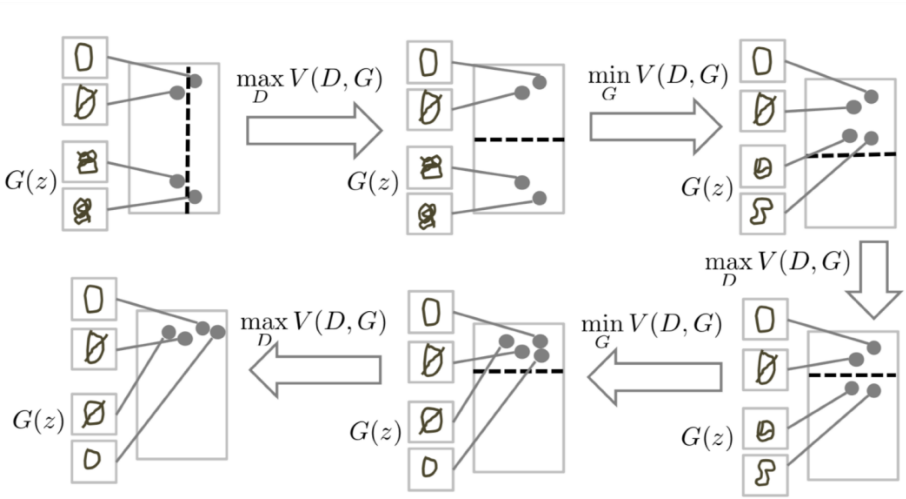
$$\max_G(\mathbb{E}_{z\sim p_z(z)}[\log(D(G(z))])$$

$D(G(z))$ should be 1

$D(G(z)) \approx 1 \Rightarrow \log D(G(z)) \approx 0$



Understanding the Objective function (cont.)



[1] https://www.slideshare.net/ckmarkohchang/generative-adversarial-networks

GANs - Interactive Demo

https://poloclub.github.io/ganlab/

GANs: **Results**

Results

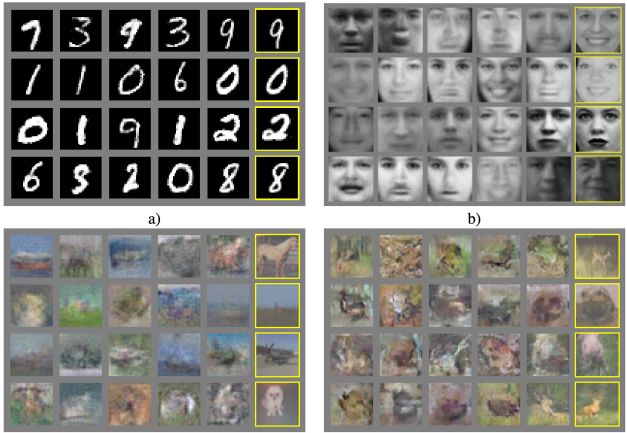
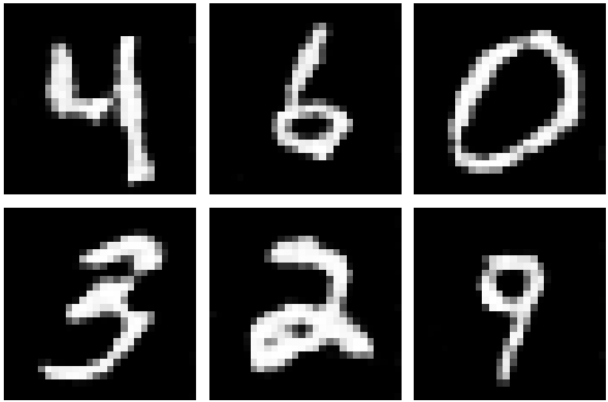


Figure from Goodfellow et al 2014, showing GAN results on various datasets.

Results (cont.)

generated_images

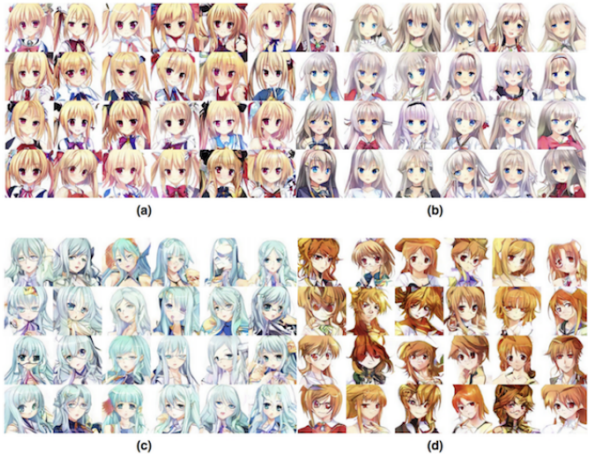


GAN generated samples for MNIST digits dataset

Results (cont.)

GAN generated samples for bedroom images

Results (cont.)



GAN generated samples for anime character faces

Problems with GANs

- Vanishing Gradients
  - When the discriminator is perfect, we are guaranteed with $D(x) = 1, \forall x \in p_{data}$ and $D(x) = 0, \forall x \in p_G$.
  - The loss function drops to zero, resulting in no gradient for updates.
  - **Solution**: Perform gradient ascent on the generator, i.e., use a different objective:

$$\max_{\theta_G} E_{x \sim p(z)} log(D_{\theta_D}(G_{\theta_G}(z)))$$

Problems with GANs (cont.)

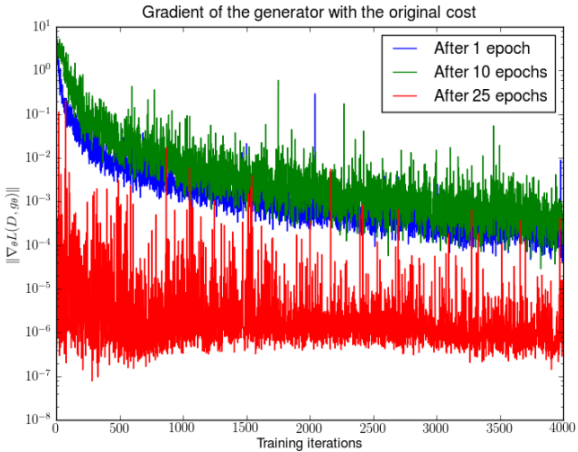


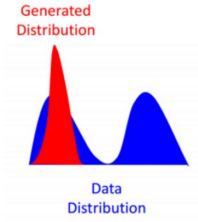
Figure 6: Vanishing gradient in GANs (Image source: Arjovsky and Bottou, 2017)

Problems with GANs (cont.)

- Difficulty in achieving Nash equilibrium
  - GANs involve training two models simultaneously to reach a Nash equilibrium in a two-player non-cooperative game. However, since each model updates its cost independently, convergence is not guaranteed.
  - For practical tips on training GANs, see "How to Train a GAN? Tips and tricks to make GANs work" by Soumith Chintala: https://github.com/soumith/ganhacks

Problems with GANs (cont.)

- Mode collapse
  - The generator aims to fool the discriminator D into classifying its outputs as real. If the generator G finds a single output that consistently fools D, it may repeatedly produce that output, leading to mode collapse.
  - Solutions to mode collapse are mostly empirical, including alternative architectures, modified GAN losses, and additional regularization terms.



Problems with GANs (cont.)

Figure 7: GAN mode collapse on MNIST digits dataset

Problems with GANs (cont.)

GANs: **More Results**

Some more GAN results

Image Inpainting. https://www.nvidia.com/en-us/research/ai-demos/

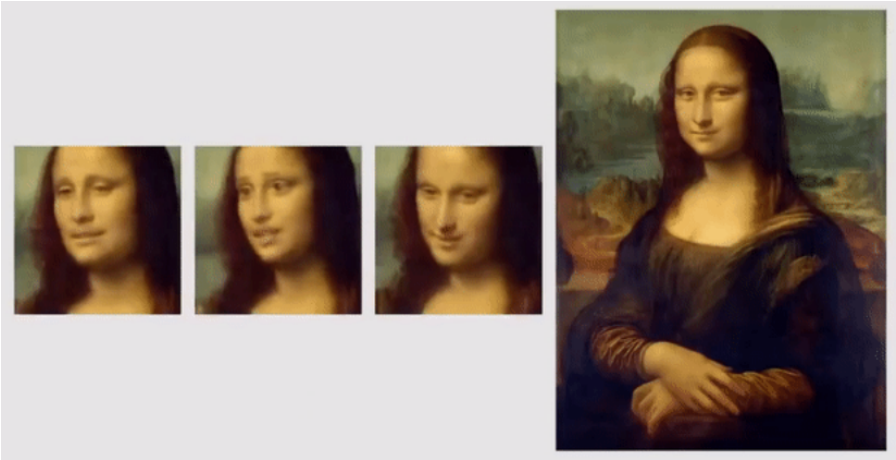
Some more GAN results (cont.)

this small bird has a pink breast and crown, and black primaries and secondaries.

this white and yellow flower have thin white petals and a round yellow stamen

Text to Image Synthesis with GANs

Some more GAN results (cont.)



Living Portraits with GANs

Some more GAN results (cont.)

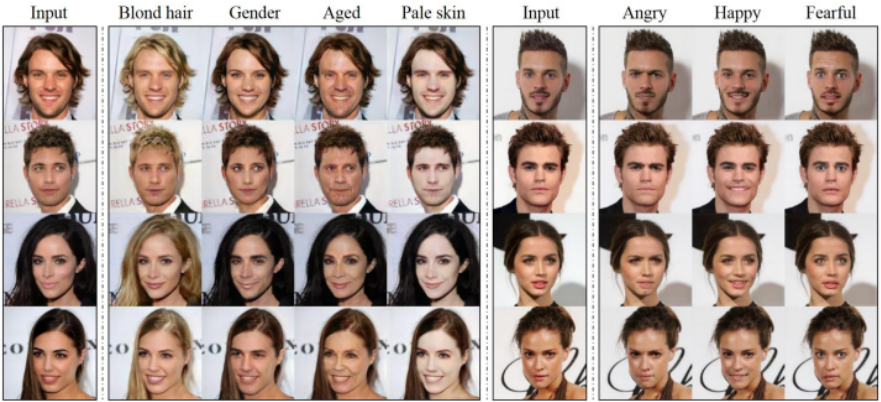


Image to Image translation in multiple domains with StyleGAN (Choi et al.)

Summary - GANs

- Instead of explicitly learning data probability distribution. Learn it implicitly.
- Train two networks jointly. Generator generates fake samples and aims to fool discriminator. Discriminator aims to discriminate between real and fake samples.
- GANs are notoriously difficult to train. Lots of tips and tricks available.
- Huge amount of research done on GANs and plenty of variations with interesting results.

GANs: **Limitations**

Limitations

- Hard to train
- Sensitive to hyperparameters
- No clear evaluation metric
- Mode collapse is common
- Often unpredictable behavior

GANs: **References**

References

Reference Slides

- Fei-Fei Li "Generative Deep Learning" CS231
- Francois Fleuret "Deep Learning" EE559
- Murtaza Taj "Deep Learning" CS437

References (Papers)

- Goodfellow et al., "Generative Adversarial Networks," 2014. arXiv:1406.2661
- Arjovsky et al., "Wasserstein GAN," 2017. arXiv:1701.07875
- Zhu et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks," 2017. arXiv:1703.10593
- Karras et al., "A Style-Based Generator Architecture for Generative Adversarial Networks," CVPR 2019. arXiv:1812.04948
- Creswell et al., "Generative Adversarial Networks: An Overview," IEEE Signal Processing Magazine, 2018. IEEE Xplore